

PEL Quick Access Topics Index

© *Pierre Rouleau* 2021-2026

Last updated on: 2026-06-10

GNU Emacs

Reference Cards

• Emacs Release History

• EmacsWiki

• Emacs project repo

➤ PEL Repo

Readme

• License

Manual

• NEWS

Discussions

Terminal Multiplexers:

GNU screen , Tmux

Command Line Scripting

Languages: bash, sh, zsh

🐉: GNU readline, ls -l, ssh

General Info

➤

Startup

➤

PEL Code

➤

• Contribute to Emacs

• Mailing Lists

• EmacsConf

This table holds links to all other PEL topic oriented PDF table files (hosted on Github Pages).

🙌 From within Emacs open this topic index PDF by typing the <f11> ? <f1> key sequence. More help topics with <f11> ? p keys.

🙌 The symbols, colour coding and various other conventions are described in the ➤Legend PDF.

Emacs

Emacs survival card

Calc

Dired

Gnus

Gnus booklet

Magit Cheatsheet

Magit Ref-card

Org

Viper

VIP

OS Desktop Key Bindings

(Bindings that don't clash with PEL)

🍏 macOS Fct Keys

🍏 macOS Keys

🍏 terminal settings

🐉Mint 20 Desktop Keys

🐉Rocky Linux 8 Desktop Keys

🐉Ubuntu 16.04 Desktop Keys

🐉 Feature Comparisons

🐉 Completion Modes Compatibility

🐉 Speedbar/iMenu Mode Compatibility

🐉 Shells/Terminals Comparisons

Prefix/Suffix & Numeric Arg Keys

ℹ ≡ Modifier Keys

ℹ ≡ Numkeypad

≡ Keys - Fn

≡ Keys - F11

≡ Keys - F12

➤PEL

ℹ Emacs Features

ℹ Emacs Manual , Guided Tour of Emacs , Emacs Lisp Manual

• Emacs Docs: Emacs, Emacs Lisp

• Mastering Emacs, Awesome-Emacs

• MELPA and GNU ELPA

The tables at right describe Emacs concepts/ features commands & key bindings. Cell background is light-blue for major mode, light-red for minor mode specifics, grey for links to sections of other tables. Cells link titles starting with ℹ are Emacs generic features, blue links are external packages. The green links are mostly PEL extensions. Emacs commands can be executed by name or bound to key sequences. They describe the commands, their arguments and the key sequences bound to them.

• Emacs Keys and Numeric Arguments

You can also: Run Command by Name

Emacs uses a concept of modes:

• Emacs Major and Minor Modes

• Major Modes

• Minor Modes

• Choosing Modes

PEL provides several key sequences to toggle minor modes.

1979 EMACS Intro memo by R.M. Stallman

ℹ Abbreviations

Debuggers🔗🔗

ℹ Grep (grep-regexp)

ℹ Man pages

ℹ Scrolling

ℹ Tab Bar

ℹ Align

ℹ Diff & Merge

ℹ Help/Info

ℹ Marking

ℹ Search/Replace

[T Templates](#)

ℹ Auto-Completion

ℹ Dired

ℹ Hide/Show

ℹ Menus

ℹ iMenu

ℹ Sessions

ℹ Text Modes

ℹ Autosave/Backup

ℹ Display - Lines

ℹ Highlight (colors)

ℹ Mode Line

ℹ start Shells/REPLs

ℹ Time Stamps

ℹ Bookmarks

ℹ Drawing

ℹ ibuffer-mode

ℹ Mouse

ℹ shell-mode

ℹ Time Tracking

ℹ Buffers

ℹ Eldoc

ℹ Indentation

ℹ Narrowing

ℹ term-mode

ℹ Tramp 🗺️

ℹ Case Conversions

ℹ Enriched Text

ℹ Input Method

ℹ Navigation

[eat-mode](#)

ℹ Transpose text

ℹ Close/Suspend

ℹ Execute Cmds

ℹ Inserting Text

ℹ Object Files

[vterm-mode](#)

[ℹℹ Treemacs](#)

ℹ Comments

ℹ Exec Shell Cmds

ℹ Key-Chords

ℹ Outline

[ℹℹ Smartparens](#)

ℹ Tree Sitter

ℹ Compilation Mode

ℹ Faces/Fonts

ℹ Keyboard Macros

ℹ Packages

ℹ Sorting

ℹ Undo/Redo/Repeat

ℹ Completion/Input

[ℹP Fast Startup](#)

ℹ Line Ending Mngt

Programming

[Speech To Text](#)

ℹ VCS-Git [ℹMagit](#)

ℹ Copyright

ℹ File Encoding

[ℹℹℹ - Lispy](#)

ℹ Project Tools

ℹ Speedbar

ℹ VCS-Mercurial

ℹ Counting

ℹ File-mngt

[Logging key strokes](#)

[ℹℹ Projectile](#)

ℹ Spell Checking

ℹ VCS-Subversion

[ℹM CUA](#)

ℹ File/Dir Variables

ℹ Recursive Edit

ℹ SyntaxCheck

ℹ Web

ℹ Cursor

ℹ Fill/Justify

ℹ Rectangles

ℹ Whitespace

ℹ Customize

ℹ Frames

ℹ Registers

ℹ Windows

ℹ Cut & Paste

[Writing Tools](#)

ℹ Xref - Cross Refs

Emacs Lisp Ref concepts

& tools

🔗 display-buffer

🔗 Hooks

🔗 - ELisp Topics

🔗 - ELisp Types

🔗 Elisp Build Tools

🔗 ERT (regr-testing)

Parsing tools, Indentation

ℹ Xref Tools:

🐉 Indentation Styles

🐉 Language Servers

🐉 Tree-sitter

🐉 Xref-Backend

🐉 Xref-Frontend

🐉 Xref-Support

[Build Tools](#)

🐉 - CMake 🔗🔗

🐉 - Make [gmake](#)

🐉 - Meson

🐉 - Ninja

🐉 - Nix

🐉 - Tup

[Data Serialization & Configuration](#)

🕒 CWL

🕒 HCL/Terraform🔗🔗

🕒 JSON 🔗🔗

🕒 PKL 🔗🔗

🕒 XML 🔗🔗

[xmake](#)

[Modelling](#)

📄 ASN.1 [asn1-mode](#)

📄 MIB [snmp-mode](#)

📄 YANG

🕒 YAML

[Other File Formats](#)

Binary, Object, Executable Files

Log Files

RFC (RFC @ Wikipedia)

SSH files 🐉ssh

ℹ Changelog Files

Config/ini/toml... Files

RPM Files 🐉 (spec file format)

[M X.509 Certificates](#)

[Hardware Description Languages](#)

🔗 - Verilog 🔗🔗

🔗 - VHDL 🔗🔗

🐉 Language Server & Tools for HDL 🔗🔗

[Lightweight Markup Languages](#)

[M AsciiDoc](#)

[M Markdown](#)

[M Org-Mode](#)

[M reStructuredText](#)

[M HamI](#) 🔗🔗 future

• Graphics Markup

[M Graphviz Dot](#)

[M MscGen](#)

[M PlantUML](#)

Programming Languages Major Modes

BEAM Programming

[Functional](#)

Javascript target

Pascal-style syntax

Lisp-like Languages

Stack Based

Curly Bracket

Java Virtual Machine

ML Family

Lisp Family [H](#) [m](#)

Scheme Dialects[H](#) [m](#)

OS App Control

Main Paradigm of Programming Languages

• Actor Model: [A](#) Array [X](#)

• Concatenative [K](#) Concurrent: [C](#)

• Domain Specific [d](#)

• Dynamic [d](#) Extensible [C](#)

• Functional: [f](#) Pure: [F](#)

• Generic [G](#) homoiconic [H](#)

• Imperative: [i](#) or no token

• Object Oriented [O](#) Procedural [P](#)

• Has Syntactic Macros: [m](#)

• Multi-paradigm [a](#) Reflective |

• System Level [S](#)

The programming languages supported by PEL are listed here in alphabetical order.

Emacs (and PEL) also provides basic support for some of the one PEL does not support and for other programming languages not listed here.

🐉 - Ada 🔗🔗 [a](#) [S](#)

🐉 - D [i](#) [f](#) [A](#)

🐉 - Gambit [f](#) [m](#)

🐉 - Janet [i](#) [f](#) [m](#)

🐉 - Pascal

Scala 🔗🔗

🐉 - Algol

🐉 - Dart [a](#) [f](#) [O](#) |

🐉 - Gerbil [f](#) [m](#) [A](#)

🐉 - Java 🔗🔗

🐉 - Perl (perl5)

[🐉 - Scheme](#) [f](#) [m](#)

🐉🍏 - AppleScript

🐉 - Eiffel 🔗🔗 [O](#) [S](#)

🐉 - GNU Guile [f](#) [m](#)

🐉 - Javascript 🔗🔗

PHP 🔗🔗 future

[🐉 - Shen](#) [f](#) [m](#)

APL 🔗🔗

Elm 🔗🔗 [F](#)

🐉 - Gleam

🐉 - Julia [m](#)

🐉 - Pike [d](#) [i](#) [O](#)

[🐉 - Seed7](#) [C](#) [O](#) [G](#) [a](#)

🐉 - Arc [f](#) [m](#)

🐉 - Elixir [C](#) [m](#) [f](#) [A](#)

🐉 - Go [S](#)

Kotlin 🔗🔗

Prolog 🔗🔗

SQL 🔗🔗

🐉 - awk [d](#)

[ℹ🐉 - Emacs Lisp](#)

Groovy 🔗🔗

🐉 - LFE [C](#) [m](#) [f](#) [A](#)

Pony 🔗🔗

🐉 - Smalltalk 🔗🔗 [O](#)

🐉 - C [S](#)

🐉 - Erlang [C](#) [f](#) [A](#)

🐉 - Haskell [F](#)

🐉 - Lua [f](#) [O](#) [P](#)

Purescript 🔗🔗 [F](#)

🐉 - Swift

C# 🔗🔗 future

🐉 - Factor [K](#) [f](#) [O](#) [m](#)

Haxe 🔗🔗

Luerl 🔗🔗

🐉 - Python [d](#) [P](#) [O](#) [f](#)

🐉 - Tcl [f](#) [i](#)

🐉 - C++ [O](#) [S](#)

FAUST 🔗🔗 future

[🐉 - Hy \(python\)](#) [m](#)

🐉 - M4

[R](#) 🔗🔗 [O](#) [P](#) [f](#) [X](#) |

🐉 - Typescript 🔗🔗

🐉 - C3 [S](#)

Fennel 🔗🔗 future

[🐉 - Modula](#)

[🐉 - Racket](#) [f](#) [m](#)

[🐉 - UNIX Shell](#)

Carbon 🔗🔗 future

[🐉 - Forth](#) [K](#)

Mojo 🔗🔗 future

ReasonML 🔗🔗

[🐉 - V](#)

🐉 - Chez [f](#) [m](#)

🐉 - Fortran

🐉 - NetRexx

🐉 - Rebol | [H](#)

Vala 🔗🔗 future

🐉 - Chibi [f](#) [m](#)

🐉 - Nim [m](#) [S](#)

Red 🔗🔗

[🐉 - Zig](#) [S](#)

🐉 - Chicken [f](#) [m](#)

[🐉 - Objective-C](#) 🔗🔗

🐉 - REXX

🐉 - Clojure [f](#) [m](#)

[🐉 - OCaml](#) [i](#) [f](#)

Rocq 🔗🔗 future

[🐉 - Common Lisp](#) [m](#)

🐉 - Odin [S](#)

🐉 - Ruby

Crystal 🔗🔗

[🐉 - Rust](#) [S](#)